

Laporan Praktikum Kontrol Cerdas

Minggu 3 Prak 15 Scikit Learn

Nama : Kuswoyo Priyokusumo
NIM : 234308076
Kelas : TKA 6C
Akun Github : <https://github.com/kuswoyop>

Abstrak

Praktikum ini mengimplementasikan konsep machine learning menggunakan Scikit-Learn yang diintegrasikan dengan OpenCV dan MediaPipe dalam sistem deteksi gesture tangan serta identifikasi pose berdiri dan duduk. Proses dimulai dari pengumpulan data citra melalui webcam, dilanjutkan dengan ekstraksi fitur menggunakan landmark tangan dari MediaPipe, kemudian dilakukan pelatihan model klasifikasi menggunakan algoritma Random Forest. Data dibagi menjadi data latih dan data uji untuk mengevaluasi performa model menggunakan metrik akurasi. Hasil implementasi menunjukkan bahwa model mampu mengklasifikasikan gesture secara real-time dengan performa yang baik. Selain itu, pada deteksi pose berdiri dan duduk digunakan pendekatan berbasis aturan dengan memanfaatkan koordinat landmark tubuh. Secara keseluruhan, praktikum ini menunjukkan bahwa integrasi pengolahan citra dan machine learning dapat diterapkan secara efektif dalam membangun sistem visi komputer sederhana berbasis Python.

Kata kunci: Machine Learning, Scikit-Learn, MediaPipe, OpenCV, Deteksi Gesture, Random Forest.

1. Pendahuluan :

Perkembangan teknologi kecerdasan buatan (Artificial Intelligence) mendorong semakin luasnya penggunaan machine learning dalam pemecahan masalah nyata yang melibatkan prediksi, klasifikasi, dan pengambilan keputusan berbasis data. Machine learning merupakan cabang ilmu komputer yang memungkinkan sistem belajar dari data untuk mengenali pola dan membuat prediksi tanpa diprogram secara eksplisit pada setiap kasusnya. Bahasa pemrograman Python menjadi pilihan utama dalam bidang ini karena sintaksisnya sederhana, kaya akan pustaka (library) pendukung, serta komunitas besar yang aktif dalam pengembangan ekosistem data science dan AI (Alfiyani dkk., 2024).

Scikit-Learn ada sebagai salah satu library machine learning yang paling banyak digunakan oleh peneliti maupun praktisi. Scikit-Learn menyediakan beragam alat untuk menyusun model prediksi melalui teknik klasifikasi, regresi, klastering, serta fungsi pra-pemrosesan dan evaluasi model yang lengkap dan konsisten. Keunggulan Scikit-Learn terletak pada kemudahan penggunaannya, dokumentasi yang kuat, serta

kompatibilitas dengan library lain seperti NumPy dan SciPy sehingga memudahkan alur pengolahan data secara terpadu(Raschka dkk., 2020).

Dalam konteks pendidikan dan penelitian di Indonesia, Scikit-Learn sering dimanfaatkan untuk memahami konsep supervised maupun unsupervised learning serta menerapkan algoritma populer seperti K-Nearest Neighbors, Linear Regression, PCA, dan K-Means dalam analisis data nyata(Fahmi, 2023). Pustaka ini tidak hanya relevan untuk tugas akademik, tetapi juga aplikasi dunia nyata di berbagai sektor seperti kesehatan, ekonomi, dan teknologi informasi karena kemampuannya memproses data dari tahap prapemrosesan hingga evaluasi performa model.

2. Tujuan :

- Memahami konsep dasar machine learning menggunakan library scikit-learn.
- Mempelajari proses pembuatan model klasifikasi untuk deteksi gesture/tampilan tubuh.
- Mengintegrasikan MediaPipe untuk ekstraksi fitur (posisi tangan/tubuh) dengan scikit-learn untuk pemrosesan data.
- Melatih model machine learning dari data yang dikumpulkan.

3. Manfaat :

- Mahasiswa mampu mengimplementasikan algoritma machine learning secara langsung.
- Meningkatkan pemahaman tentang alur kerja machine learning (pengumpulan data, pelatihan, evaluasi).
- Memperoleh keterampilan dalam pengolahan dan analisis data berbasis Python.

4. Hasil Program :

A. Pengumpulan data menggunakan webcam



```
5. import os
from time import sleep
import cv2
DATA_DIR = 'C:\SEMESTER6\prak.kontrol.cerdas\mingguk3\DATA'
```

```

if not os.path.exists(DATA_DIR):
    os.makedirs(DATA_DIR)
number_of_classes = 2
dataset_size = 100
cap = cv2.VideoCapture(0)

if not cap.isOpened():
    print("Camera tidak bisa dibuka")
    exit()

for j in range(number_of_classes):
    if not os.path.exists(os.path.join(DATA_DIR, str(j))):
        os.makedirs(os.path.join(DATA_DIR, str(j)))
    print('Collecting data for class {}'.format(j))

    while True:
        ret, frame = cap.read()
        if not ret:
            print("Gagal membaca frame")
            break
        cv2.putText(frame, 'Ready? Press "Q" !', (100, 50),
cv2.FONT_HERSHEY_SIMPLEX, 1.3, (0, 255, 0), 3)
        cv2.imshow('frame', frame)
        if cv2.waitKey(15) == ord('q'):
            break

        counter = 0
        while counter < dataset_size:
            ret, frame = cap.read()
            cv2.imshow('frame', frame)
            cv2.waitKey(10)
            cv2.imwrite(os.path.join(DATA_DIR, str(j),
'{}.jpg'.format(counter)), frame)
            counter += 1

cap.release()
cv2.destroyAllWindows()

```

B. Ekstraksi data dari dataset

Name	Date modified	Type	Size
DATA	26/02/2026 20:49	File folder	
fotohasilprogram	27/02/2026 07:01	File folder	
data.pickle	26/02/2026 21:03	PICKLE File	38 KB
model.p	26/02/2026 21:17	P File	37 KB
statistikprogram.py	27/02/2026 00:00	Python File	2 KB

```

6. import os
import pickle
import mediapipe as mp
import cv2

mp_hands = mp.solutions.hands
hands = mp_hands.Hands(static_image_mode=True, max_num_hands=1)

DATA_DIR = 'C:\SEMESTER6\prak.kontrol.cerdas\mingguke3\DATA'
data = []
labels = []

```

```

for dir_ in os.listdir(DATA_DIR):
    for img_path in os.listdir(os.path.join(DATA_DIR, dir_)):
        data_aux = []
        x_ = []
        y_ = []
        img = cv2.imread(os.path.join(DATA_DIR, dir_, img_path))
        img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
        results = hands.process(img_rgb)

        if results.multi_hand_landmarks:
            for hand_landmarks in results.multi_hand_landmarks:
                for i in range(len(hand_landmarks.landmark)):
                    x = hand_landmarks.landmark[i].x
                    y = hand_landmarks.landmark[i].y
                    x_.append(x)
                    y_.append(y)

                for i in range(len(hand_landmarks.landmark)):
                    x = hand_landmarks.landmark[i].x
                    y = hand_landmarks.landmark[i].y
                    data_aux.append(x - min(x_))
                    data_aux.append(y - min(y_))

            data.append(data_aux)
            labels.append(dir_)

f = open('C:\SEMESTER6\prak.kontrol.cerdas\mingguke3\data.pickle',
        'wb')
pickle.dump({'data': data, 'labels': labels}, f)
f.close()

```

C. Pelatihan dan pengujian

Name	Date modified	Type	Size
DATA	26/02/2026 20:49	File folder	
fotohasilprogram	27/02/2026 07:01	File folder	
data.pickle	26/02/2026 21:03	PICKLE File	38 KB
model.p	26/02/2026 21:17	P File	37 KB
latihanBersama...	27/02/2026 00:22	Python File	2 KB

```

7. import pickle
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
import numpy as np

data_dict =
pickle.load(open('C:\SEMESTER6\prak.kontrol.cerdas\mingguke3\data.pickle', 'rb'))
data = np.asarray(data_dict['data'])
labels = np.asarray(data_dict['labels'])
x_train, x_test, y_train, y_test = train_test_split(data, labels,
                                                    test_size=0.1, shuffle=True, stratify=labels)
model = RandomForestClassifier()
model.fit(x_train, y_train)
y_predict = model.predict(x_test)
score = accuracy_score(y_predict, y_test)

```

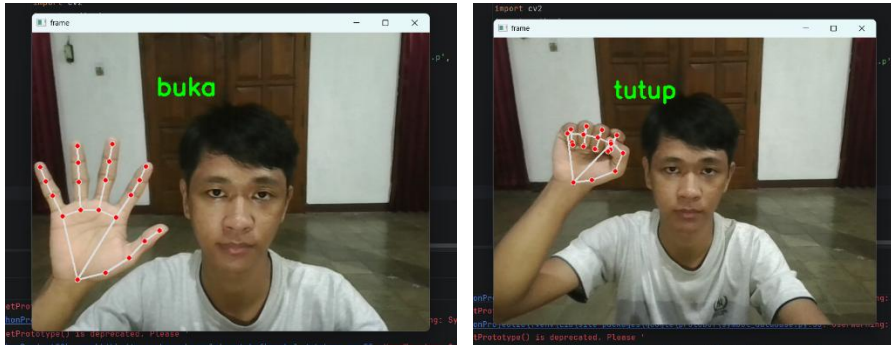
```

print('{}% of samples were classified correctly !'.format(score *
100))

f = open('C:\SEMESTER6\prak.kontrol.cerdas\mingguke3\model.p', 'wb')
pickle.dump({'model.p': model}, f)
f.close()

```

D. Implementasi



```

8. import pickle
import cv2
import mediapipe as mp
import numpy as np

model_dict =
pickle.load(open(r'C:\SEMESTER6\prak.kontrol.cerdas\mingguke3\model.p
', 'rb'))
model = model_dict['model.p']

cap = cv2.VideoCapture(0)

mp_hands = mp.solutions.hands
mp_drawing = mp.solutions.drawing_utils

hands = mp_hands.Hands(max_num_hands=1)

labels_dict = {0:'buka', 1:'tutup'}

while True:
    data_aux = []
    x_ = []
    y_ = []

    ret, frame = cap.read()
    if not ret:
        break

    frame_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    results = hands.process(frame_rgb)

    if results.multi_hand_landmarks:
        for hands_landmarks in results.multi_hand_landmarks:
            mp_drawing.draw_landmarks(
                frame,
                hands_landmarks,
                mp_hands.HAND_CONNECTIONS,
            )

            for lm in hands_landmarks.landmark:

```

```

x_.append(lm.x)
y_.append(lm.y)

for lm in hands_landmarks.landmark:
    data_aux.append(lm.x - min(x_))
    data_aux.append(lm.y - min(y_))

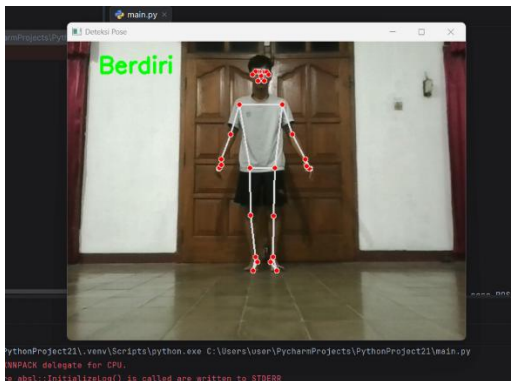
prediction = model.predict([np.asarray(data_aux)])
predicted_character = labels_dict[int(prediction[0])]
cv2.putText(frame, predicted_character, (200, 100),
cv2.FONT_HERSHEY_SIMPLEX, 1.3, (0, 255, 0), 3)

cv2.imshow('frame', frame)
if cv2.waitKey(1) & 0xFF == ord('q'):
    break

cap.release()
cv2.destroyAllWindows()

```

LATIHAN B. Menggunakan mediapipe pose buatlah program untuk mendeteksi seseorang apakah berdiri atau duduk.



```

import cv2
import mediapipe as mp
import numpy as np

mp_pose = mp.solutions.pose
pose = mp_pose.Pose()
mp_draw = mp.solutions.drawing_utils
cap = cv2.VideoCapture(0)

while True:
    ret, frame = cap.read()
    if not ret:
        break

    frame = cv2.flip(frame, 1)
    rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    results = pose.process(rgb)
    text = "Tidak terdeteksi"
    if results.pose_landmarks:
        mp_draw.draw_landmarks(frame, results.pose_landmarks,
mp_pose.POSE_CONNECTIONS)
        landmarks = results.pose_landmarks.landmark

        hip = landmarks[mp_pose.PoseLandmark.LEFT_HIP.value]

```

```

        knee = landmarks[mp.pose.PoseLandmark.LEFT_KNEE.value]
        ankle = landmarks[mp.pose.PoseLandmark.LEFT_ANKLE.value]

        if knee.y > hip.y:
            text = "Berdiri"
        else:
            text = "Duduk"

        cv2.putText(frame, text, (50, 50), cv2.FONT_HERSHEY_SIMPLEX, 1.2,
(0, 255, 0), 3)
        cv2.imshow("Deteksi Pose", frame)
        if cv2.waitKey(1) & 0xFF == ord('q'):
            break
    cap.release()
    cv2.destroyAllWindows()

```

6. Analisis Program :

Pada tahap awal, data diperoleh melalui webcam dengan bantuan OpenCV, lalu setiap citra disimpan ke dalam folder sesuai kategori yang telah ditetapkan. Pemisahan folder berdasarkan kelas mempermudah proses pelabelan karena nama direktori secara otomatis berfungsi sebagai label. Penentuan jumlah kelas serta jumlah data per kelas juga membuat distribusi data lebih terkontrol, sehingga keseimbangan antar kelas dapat dijaga dan potensi bias saat pelatihan bisa ditekan.

Setelah data terkumpul, proses dilanjutkan dengan ekstraksi fitur menggunakan MediaPipe Hands. Setiap gambar dianalisis untuk mendeteksi 21 titik landmark tangan. Nilai koordinat x dan y dari tiap titik kemudian dinormalisasi dengan mengurangi nilai minimum pada masing-masing sumbu. Cara ini membuat fitur menjadi relatif terhadap posisi tangan di dalam frame, sehingga model tidak terlalu peka terhadap perubahan letak objek. Hasil ekstraksi disimpan dalam file pickle yang berisi data dan label, sehingga proses pelatihan berikutnya dapat dilakukan tanpa perlu mengulang tahap ekstraksi. Pemisahan ini menjadikan alur eksperimen lebih efisien dan terorganisir.

Pada tahap pelatihan, dataset dibagi menjadi data latih dan data uji menggunakan `train_test_split` dengan metode stratifikasi agar proporsi tiap kelas tetap seimbang. Model yang diterapkan adalah Random Forest Classifier, yaitu algoritma berbasis kumpulan pohon keputusan. Algoritma ini dipilih karena mampu mengolah data numerik dengan baik, cukup stabil terhadap overfitting pada dataset berukuran kecil maupun menengah, serta tidak memerlukan proses prapengolahan yang rumit. Kinerja model diukur melalui nilai akurasi yang menunjukkan persentase prediksi benar terhadap data uji. Nilai tersebut memberikan gambaran tentang kemampuan model dalam mengenali data baru. Setelah proses pelatihan selesai, model disimpan untuk digunakan pada tahap implementasi.

Bagian implementasi menampilkan integrasi antara MediaPipe dan model dari Scikit-Learn dalam sistem real-time. Setiap frame yang ditangkap webcam diproses dengan prosedur ekstraksi fitur yang sama seperti saat pelatihan, kemudian hasilnya digunakan untuk memprediksi jenis gesture. Prediksi tersebut ditampilkan langsung pada layar dalam bentuk teks. Kesamaan metode ekstraksi antara tahap pelatihan dan penggunaan langsung menjadi kunci agar performa tetap konsisten. Hal ini

menunjukkan bahwa model tidak hanya efektif pada data yang tersimpan, tetapi juga dapat bekerja secara interaktif.

Pada latihan tambahan dengan MediaPipe Pose, pendekatan yang diterapkan tidak menggunakan pelatihan model, melainkan aturan sederhana berbasis perbandingan posisi landmark tubuh. Dengan mendeteksi titik seperti pinggul, lutut, dan pergelangan kaki, sistem menentukan kondisi berdiri atau duduk berdasarkan hubungan koordinat tertentu. Metode berbasis aturan ini memang lebih sederhana, tetapi cukup efektif untuk kasus yang spesifik dan tidak terlalu kompleks. Perbandingan ini memperlihatkan perbedaan karakter antara pendekatan machine learning yang fleksibel dalam mengenali pola beragam dan pendekatan rule-based yang lebih ringan untuk kebutuhan terbatas.

7. **Kesimpulan :**

Berdasarkan rangkaian praktikum yang telah dilakukan, dapat disimpulkan bahwa integrasi OpenCV, MediaPipe, dan Scikit-Learn berhasil membentuk sistem deteksi gesture tangan yang berjalan secara real-time dengan alur kerja machine learning yang lengkap, mulai dari pengumpulan data, ekstraksi fitur landmark, pelatihan model menggunakan Random Forest, hingga tahap implementasi dan evaluasi akurasi. Model mampu mengklasifikasikan gesture sesuai data latih dengan performa yang baik, menunjukkan bahwa proses prapemrosesan dan konsistensi ekstraksi fitur berperan penting terhadap hasil prediksi. Selain itu, pada latihan deteksi berdiri dan duduk menggunakan MediaPipe Pose, diperoleh pemahaman bahwa pendekatan berbasis aturan sederhana juga dapat digunakan secara efektif untuk kasus tertentu tanpa perlu pelatihan model. Secara keseluruhan, praktikum ini memperkuat pemahaman konsep machine learning pipeline sekaligus memberikan pengalaman langsung dalam membangun aplikasi visi komputer berbasis kecerdasan buatan.

8. **Saran :**

Sebagai saran jumlah dataset perlu diperbanyak dan divariasikan agar model lebih akurat dalam berbagai kondisi. Perbandingan dengan algoritma lain serta penambahan evaluasi seperti confusion matrix dapat memberikan analisis performa yang lebih lengkap. Untuk deteksi pose, pendekatan berbasis machine learning bisa dicoba agar hasil lebih stabil. Selain itu, peningkatan performa dan tampilan antarmuka akan membuat sistem lebih optimal dan siap dikembangkan lebih lanjut.

9. **Referensi :**

- Alfiyani, R., Susanto, A., Rezika, W. Y., Wicaksono, R. E., & Mudmainah, P. H. W. (2024). Analisis Statik dan Dinamik Struktur Transduser pada Low-Cost Dynamometer untuk Mengukur Gaya Potong Proses Bubut Orthogonal. *Jurnal Rekayasa Sistem Industri*, 13(1), 107–116. <https://doi.org/10.26593/jrsi.v13i1.6746.107-116>
- Fahmi, M. N. (2023). Implementasi Machine Learning menggunakan Python Library: Scikit-Learn (Supervised dan Unsupervised Learning). *Sains Data Jurnal Studi*

Matematika dan Teknologi, 1(2), 87–96.

<https://doi.org/10.52620/sainsdata.v1i2.31>

Raschka, S., Patterson, J., & Nolet, C. (2020). Machine Learning in Python: Main Developments and Technology Trends in Data Science, Machine Learning, and Artificial Intelligence. *Information*, 11(4), 193.
<https://doi.org/10.3390/info11040193>

Salehi, M. M., & Zarei, H. (2025). A Review of the Structure and Application of Scikit-Learn Datasets in Machine Learning Model Development. *International Journal of Operations Research and Artificial Intelligence*, 1(2), 88–98.
<https://doi.org/10.48314/ijorai.v1i2.64>